

Açık Kaynak Yazılımda QMOOD Tabanlı Tasarım Kalitesinin Sürüm Bazlı Evrimi: jsoup Üzerine Bir Durum Çalışması

Okan GÜMÜŞ, 255129043

Yazılım Mühendisliği Kocaeli Üniversitesi — Yazılım Mimarisi ve Tasarım Yöntemleri

GitHub: <https://github.com/okngms/qmood-jsoup-analizi>

Özet—Yazılım sistemleri, değişen gereksinimler nedeniyle sürekli evrim geçirir; bu evrim sırasında tasarım kalitesinin nasıl değiştiğinin nesnel olarak izlenmesi, bakım maliyetlerinin ve teknik borcun yönetimi açısından kritik önemdedir. Bu çalışmada, açık kaynaklı Java HTML ayrıştırıcısı jsoup'un birbirini izleyen on sürümü (1.13.1–1.22.1), Bansiya ve Davis tarafından önerilen QMOOD (Quality Model for Object-Oriented Design) modeli temel alınarak incelenmiştir. Her sürümün kaynak kodundan CK aracı ile nesne yönelimli tasarım metrikleri çıkarılmış, bu metriklerden QMOOD tasarım özellikleri ve altı yüksek düzey kalite niteliği (yeniden kullanılabilirlik, esneklik, anlaşılabilirlik, işlevsellik, genişletilebilirlik, etkinlik) ile Toplam Kalite İndeksi (TQI) hesaplanmıştır. Bulgular, tasarım boyutu, bağlaşım ve sınıf karmaşıklığının düzenli olarak artarken kapsülleme ve kohezyonun gerilediğini; TQI'nin görünürde yükselmesine rağmen bu yükselişin büyük ölçüde boyut bağımlı terimlerden kaynaklandığını, gerçek bakım göstergelerinin ise bozulduğunu ortaya koymaktadır. Bu örüntü, Lehman'ın artan karmaşıklık yasası ve yazılım yaşlanması kavramıyla tutarlı bir teknik borç birikimine işaret etmektedir. Ayrıca dört büyük dil modeliyle (ChatGPT, Gemini, DeepSeek ve Claude Sonnet 4.6) yapılan bağımsız değerlendirmeler, anlaşılabilirliğin düşüşü ve işlevselliğin artışı konusunda QMOOD ile örtüşmüştür; modellerin çoğu yeniden kullanılabilirlik gibi boyut bağımlı niteliklerde QMOOD'un aksine bozulma öngörürken, QMOOD ağırlıklandırmasını sayısal olarak uygulayan model formül tabanlı sonuçlarla örtüşerek ayrışmanın yönetsel kaynağını açığa çıkarmıştır.

Anahtar Kelimeler—yazılım kalitesi, QMOOD, nesne yönelimli metrikler, yazılım evrimi, teknik borç, CK metrikleri, jsoup.

I. GİRİŞ

Yazılım, çevresindeki dünya ile etkileşim hâlinde olan ve bu nedenle sürekli değişime zorlanan bir üründür. Lehman, gerçek dünyada kullanılan (E-tipi) programların kullanışlı kalabilmek için kesintisiz biçimde değiştirilmesi gerektiğini ve her değişimle birlikte iç karmaşıklığın, bu karmaşıklık azaltmak için özel çaba harcanmadıkça arttığını ifade etmiştir [1]. Parnas ise bu olguyu “yazılım yaşlanması” olarak adlandırmış ve tasarımın zaman içinde aşınmasının kaçınılmaz olduğunu vurgulamıştır [2]. Dolayısıyla bir yazılımın ardışık sürümleri boyunca tasarım kalitesinin nasıl seyrettiğini nesnel ölçütlerle izlemek, hem bakım maliyetlerini öngörmek hem de teknik borcu yönetmek açısından değerlidir.

Nesne yönelimli tasarımın kalitesini ölçmek için literatürde çeşitli metrik kümeleri ve hiyerarşik kalite modelleri önerilmiştir. Chidamber ve Kemerer'in metrik kümesi (CK) bağlaşım, kohezyon ve kalıtım gibi yapısal özellikleri niceliklendirir [3]. Ancak bu düşük düzey metrikler, tek başlarına “yeniden kullanılabilirlik” veya “esneklik” gibi üst düzey kalite niteliklerini doğrudan ifade etmez. Bu boşluğu doldurmak üzere Bansiya ve Davis, tasarım metriklerini kalite niteliklerine bağlayan hiyerarşik QMOOD modelini geliştirmiştir [4].

Bu çalışmanın amacı, açık kaynaklı jsoup kütüphanesinin on ardışık sürümünü QMOOD modeli temelinde analiz ederek tasarım kalitesinin sürümler arası evrimini ölçmek ve yorumlamaktır. Çalışmanın katkıları şunlardır: (i) CK aracıyla kaynak koddan elde edilen ham metriklerin QMOOD tasarım özelliklerine ve kalite niteliklerine dönüştürülmesine ilişkin uçtan uca, yeniden üretilebilir bir iş akışı; (ii) on sürüm boyunca kalite değişiminin ve teknik borç belirtilerinin niceliksel olarak ortaya konması; (iii) bulguların yazılım evrimi ve teknik borç yazını ışığında yorumlanması.

II. İLGİLİ ÇALIŞMALAR

A. Nesne Yönelimli Tasarım Metrikleri

Chidamber ve Kemerer, ölçüm kuramına dayanan altı metrikten (WMC, DIT, NOC, CBO, RFC, LCOM) oluşan ve sınıflar arası bağlaşımı, sınıf içi kohezyonu, karmaşıklık ve kalıtım hiyerarşisinin derinliğini niceliklendiren bir kümeyi önermiştir [3]; bu kümedeki WMC ölçütü, McCabe'nin çevrimsel karmaşıklık ölçümünü temel alır [5]. Li ve Henry, nesne yönelimli metriklerin bakım yapabilirliği öngörmeye kullanılabileceğini göstermiş ve MPC ile DAC gibi ek bağlaşım ölçütleri tanımlamıştır [6]. Basili, Briand ve Melo, bu metriklerin hata eğilimi gibi dışsal kalite göstergeleriyle istatistiksel olarak ilişkili olduğunu görgül biçimde doğrulamış [7]; Tang, Kao ve Chen de benzer görgül bulgular sunmuştur [8]. Bu metrikler, sonraki kalite modellerinin temel yapı taşlarını oluşturmuştur.

B. Hiyerarşik Kalite Modelleri ve QMOOD

Dromey, ürün özelliklerini ISO 9126 kalite nitelikleriyle ilişkilendiren genel bir yazılım ürünü kalite modeli önermiştir [9]. Bansiya ve Davis, Dromey'nin yaklaşımını genişleterek QMOOD modelini geliştirmiştir [4]. QMOOD, dört düzeyli bir hiyerarşidir: tasarım nitelikleri (L1), tasarım özellikleri (L2), tasarım metrikleri (L3) ve tasarım bileşenleri (L4). Model, kapsülleme, bağlaşım ve kohezyon gibi on bir tasarım özelliğini; yeniden kullanılabilirlik, esneklik, anlaşılabilirlik, işlevsellik, genişletilebilirlik ve etkinlik olmak üzere altı kalite niteliğine ağırlıklı doğrusal denklemlerle bağlar. Bu çalışmada da bu altı nitelik ve bunların toplamı olan TQI esas alınmıştır.

C. Yazılım Evrimi ve Teknik Borç

Lehman'ın yazılım evrimi yasaları, E-tipi sistemlerin sürekli değiştiğini, değiştikçe karmaşıklığının arttığını ve kalitesinin özel çaba harcanmadıkça gerilediğini öne sürer [1]; bu yasalar sonradan görgül verilerle güncellenmiştir [10]. Parnas, tasarımın zamanla bozulmasını “yazılım yaşlanması” olarak kavramsallaştırır [2]. Cunningham, gereksinimleri

hızlı karşılamak için verilen ödünlerin ilerideki bakım maliyetini artırdığını anlatmak üzere “teknik borç” eğretilmesini ortaya atmıştır [11]. Kruchten ve arkadaşları bu eğretilmeyi kuramsal bir çerçeveye oturtmuş [12], Li, Avgeriou ve Liang ise teknik borç ve yönetimi üzerine sistematik bir haritalama çalışması sunmuştur [13]. Bu çalışmada bağlaşımın artması ve kohezyonun azalması gibi belirtiler, teknik borç birikiminin nesnel göstergeleri olarak yorumlanmaktadır.

III. YÖNTEM

A. İnceleme Nesnesi

İnceleme nesnesi olarak, açık kaynaklı (MIT lisanslı), olgun ve yaygın kullanılan bir Java HTML ayrıştırma kütüphanesi olan jsoup seçilmiştir [14]. jsoup'un saf nesne yönelimli yapısı, çok sayıda sürümünün bulunması ve kaynak kodunun kamuya açık olması, sürüm bazlı bir evrim çalışması için uygun bir zemin oluşturmaktadır. Analize, 1.13.1 sürümünden 1.22.1 sürümüne kadar yıllara yayılmış on ardışık kararlı sürüm dâhil edilmiştir.

B. Metrik Çıkarımı

Her sürümün kaynak kodu resmi depodan indirilmiş ve sınıf düzeyi metrikler, statik analizle çalışan ve derlemeye gerek duymayan CK aracı ile çıkarılmıştır [15]. Araç, her sınıf için CBO, DIT, NOC, RFC, WMC, LCOM gibi CK metriklerinin yanı sıra metot ve alan sayıları, görünürlük dağılımları gibi ek ölçütleri “class.csv” biçiminde üretmiştir. Yalnızca ana kaynak (src/main/java) dizini analiz edilerek test sınıflarının ölçümleri etkilemesi önlenmiştir.

C. QMOOD Tasarım Özelliklerine Eşleme

CK çıktısından, QMOOD'un on bir tasarım özelliği proje düzeyinde toplanmıştır. Tasarım boyutu (DSC) sınıf sayısı, karmaşıklık (NOM) sınıf başına ortalama metot sayısı, mesajlaşma (CIS) ortalama public metot sayısı, bağlaşım (DCC) ortalama CBO, soyutlama (ANA) ortalama kalıtım derinliği ve kapsülleme (DAM) gizli alanların toplam alanlara oranı olarak doğrudan ya da türetilerek elde edilmiştir. QMOOD'un özgün tanımlarında kohezyon (CAM) bir sınıftaki metotların parametre tiplerinin örtüşmesine, kompozisyon (MOA) kullanıcı tanımlı tipteki alan bildirimlerine, kalıtım (MFA) kalıtılan metotların erişilebilir metotlara oranına dayanır [4]. Bu üç özellik ile çokbiçimlilik (NOP), CK çıktısında doğrudan bulunmadığından, sırasıyla normalize LCOM ölçütüne (LCOM*) [16] dayanan kohezyon ($1 - \text{LCOM}^*$), örnek alan

sayısı, kalıtım derinliği tabanlı oran ve geçersiz kılınabilir metot sayısı kullanılarak yaklaşık biçimde hesaplanmıştır. Bu yaklaşımların kurma geçerliliğine etkisi VI. Bölümde tartışılmaktadır.

D. Normalizasyon

Tasarım özelliklerinin ölçekleri birbirinden büyük farklılık gösterdiğinden (örneğin DSC yüzlerce iken DAM 0–1 aralığındadır), karşılaştırılabilirlik için her özellik ilk sürümün (1.13.1) değerine bölünerek normalize edilmiştir. Böylece baz sürüm tüm özellikler için 1,0 değerini alır ve sonraki sürümler bağıl değişimi yansıtır. Bu yaklaşım, QMOOD değerlerinin mutlak değil bağıl olarak yorumlanması gerektiği ilkesiyle uyumludur.

E. Kalite Niteliklerinin Hesaplanması

Normalize tasarım özellikleri, QMOOD'un altı kalite niteliği denkleminde yerine konularak her sürüm için nitelik değerleri hesaplanmıştır. Örneğin yeniden kullanılabilirlik = $0,5 \cdot \text{DSC} - 0,25 \cdot \text{DCC} + 0,25 \cdot \text{CAM} + 0,5 \cdot \text{CIS}$; etkinlik = $0,2 \cdot (\text{ANA} + \text{DAM} + \text{MOA} + \text{MFA} + \text{NOP})$ biçimindedir [4]. Altı niteliğin toplamı Toplam Kalite İndeksi (TQI) olarak alınmıştır. Hazır kalite skorları kullanılmamış, tüm hesaplamalar ham metriklerden Python ortamında yeniden üretilebilir biçimde yapılmıştır.

IV. BULGULAR

Tablo I, seçili tasarım özelliklerinin sürümler boyunca ham değerlerini; Tablo II ise normalize girdilerle hesaplanan kalite niteliklerini ve TQI'yi göstermektedir. Tasarım boyutu (DSC) 242'den 290'a (%19,8), bağlaşım (DCC) 4,82'den 5,52'ye (%14,7), sınıf başına metot sayısı (NOM) 6,09'dan 7,24'e (%18,8) yükselmiştir. Buna karşılık kapsülleme (DAM) 0,752'den 0,654'e (%13,1) ve kohezyon (CAM) 0,853'ten 0,788'e (%7,7) gerilemiştir.

Kalite nitelikleri düzeyinde işlevsellik (+%32,6), yeniden kullanılabilirlik (+%12,8) ve esneklik (+%11,8) belirgin biçimde artarken, anlaşılabilirlik baz sürümüne göre yaklaşık %30 oranında kötüleşmiştir (değer -0,99'dan -1,29'a inmiştir); QMOOD'da anlaşılabilirlik negatif ağırlıklı bir nitelik olduğundan, değer negatif yönde büyümesi (mutlak değerce artması) tasarımın anlaşılmasının zorlaştığını ve kalitenin gerilediğini gösterir. Genişletilebilirlik ve etkinlik görece olarak durağan kalmıştır. TQI genel olarak 4,01'den 4,36'ya yükselmiş; ancak 1.20.1 sürümünde 4,20'ye gerileyip ardından toparlanan yerel bir düşüş gözlenmiştir. Şekil 1–6 bu eğilimleri görselleştirmektedir.

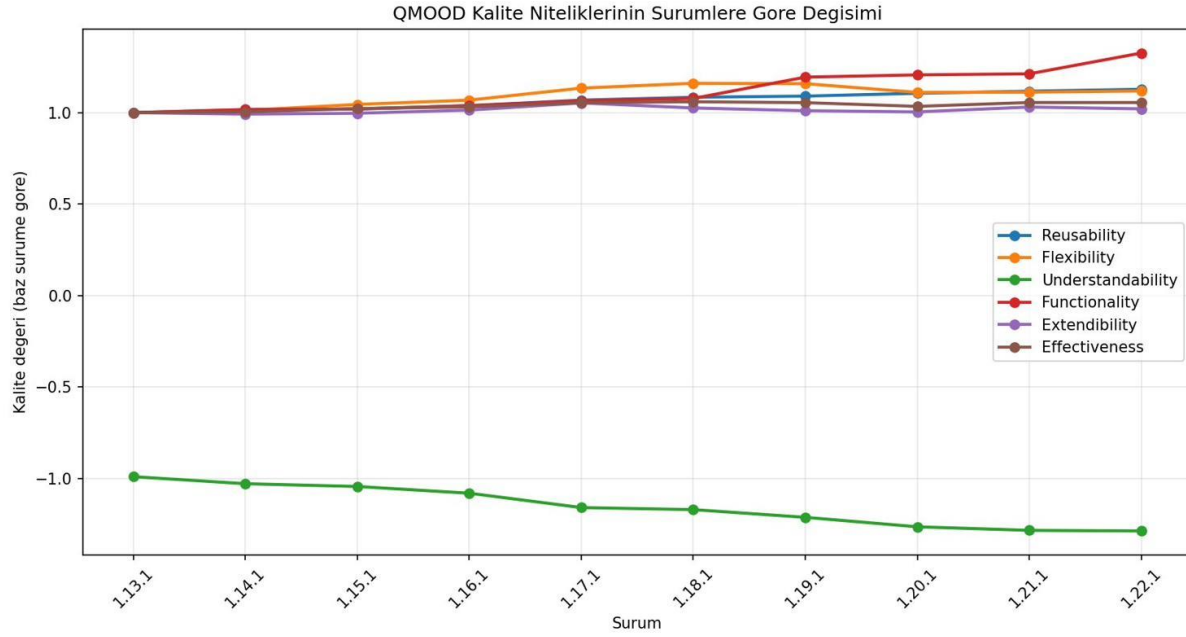
TABLO I Seçili QMOOD tasarım özelliklerinin sürümlere göre ham değerleri

Sürüm	DSC	DCC	CAM	DAM	MOA	NOP	NOM
1.13.1	242	4.82	0.853	0.752	0.905	2.98	6.09
1.14.1	246	4.98	0.852	0.744	0.919	3.07	6.28
1.15.1	250	5.01	0.842	0.75	0.976	3.08	6.35
1.16.1	253	5.05	0.835	0.741	0.992	3.2	6.52
1.17.1	261	5.05	0.806	0.698	1.077	3.39	6.79
1.18.1	265	5.15	0.806	0.712	1.113	3.44	6.89
1.19.1	270	5.29	0.799	0.669	1.144	3.45	6.99
1.20.1	275	5.44	0.795	0.623	1.091	3.48	7.18

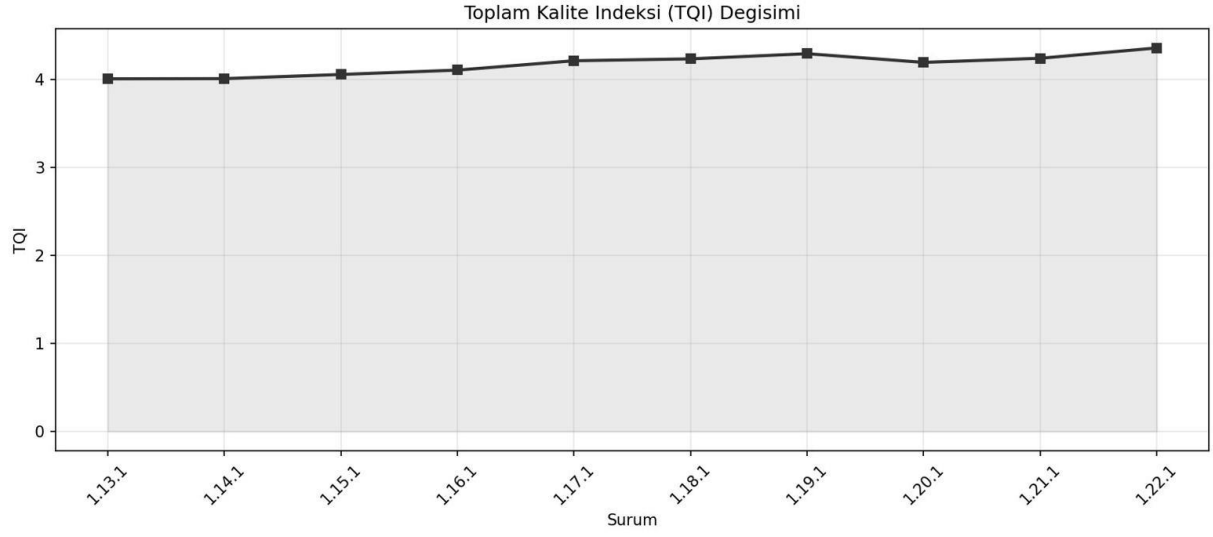
Sürüm	DSC	DCC	CAM	DAM	MOA	NOP	NOM
1.21.1	285	5.53	0.792	0.645	1.095	3.46	7.19
1.22.1	290	5.52	0.788	0.654	1.103	3.45	7.24

TABLO II QMOOD kalite nitelikleri ve Toplam Kalite İndeksi (baz sürüme göre normalize)

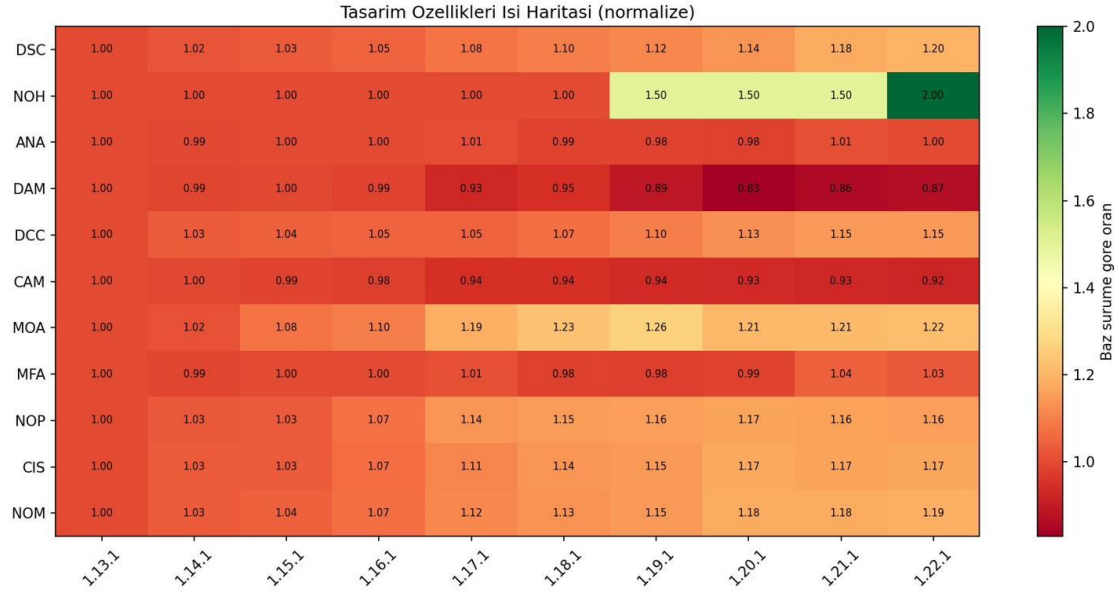
Sürüm	Reus.	Flex.	Underst.	Funct.	Extend.	Effect.	TQI
1.13.1	1	1	-0.99	1	1	1	4.01
1.14.1	1.016	1.012	-1.029	1.017	0.992	1.004	4.01
1.15.1	1.019	1.045	-1.044	1.02	0.997	1.022	4.06
1.16.1	1.038	1.068	-1.08	1.038	1.014	1.032	4.11
1.17.1	1.068	1.134	-1.159	1.065	1.053	1.054	4.22
1.18.1	1.084	1.16	-1.17	1.078	1.025	1.059	4.24
1.19.1	1.09	1.158	-1.212	1.194	1.01	1.055	4.3
1.20.1	1.106	1.111	-1.264	1.206	1.004	1.034	4.2
1.21.1	1.117	1.112	-1.284	1.212	1.031	1.055	4.24
1.22.1	1.128	1.118	-1.287	1.326	1.021	1.055	4.36



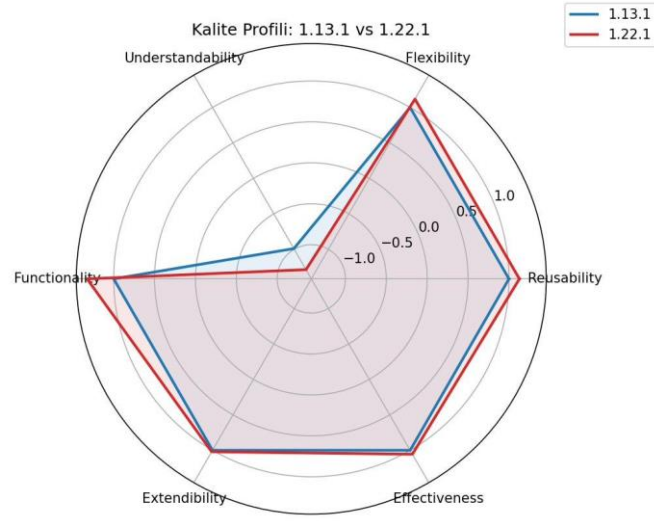
Şekil 1. Kalite niteliklerinin sürümlere göre değişimi.



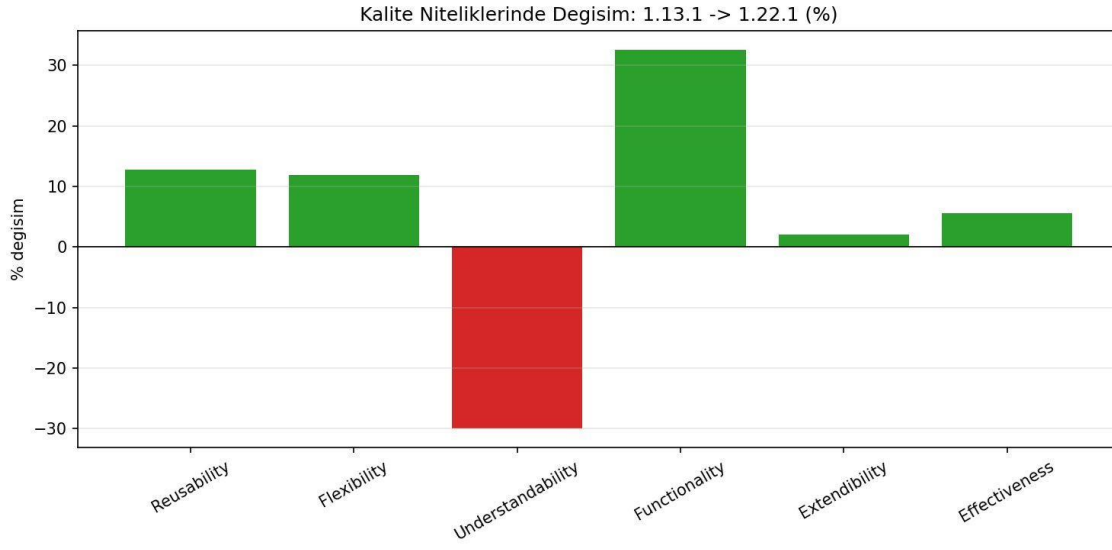
Şekil 2. Toplam Kalite İndeksi (TQI) değişimi.



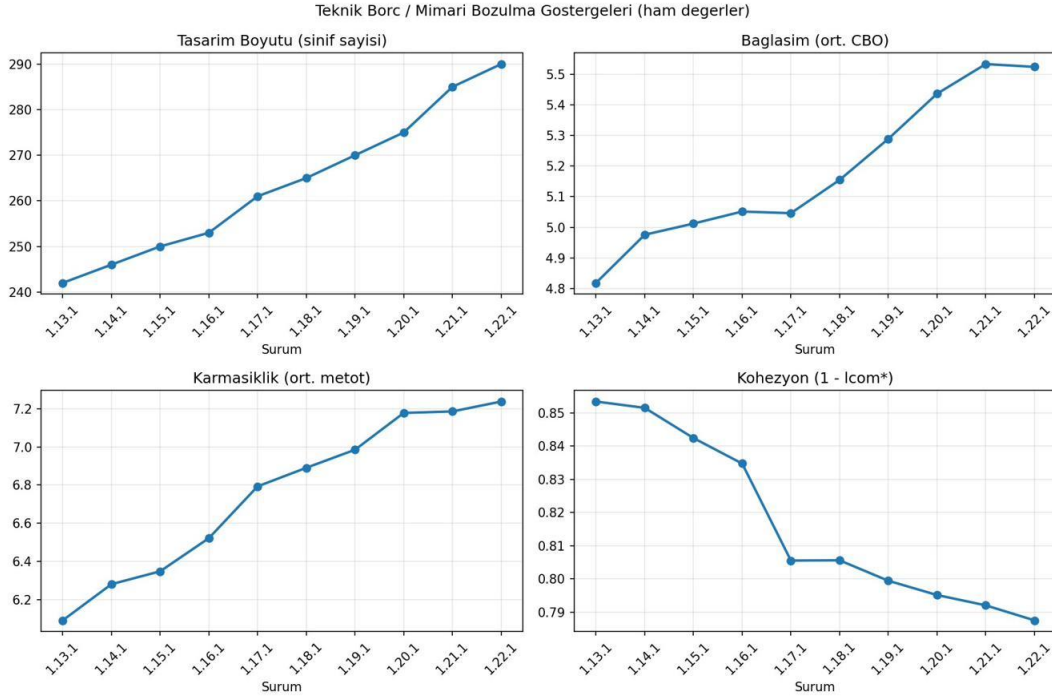
Şekil 3. Tasarım özelliklerinin normalize ısı haritası.



Şekil 4. İlk ve son sürümün kalite profili karşılaştırması.



Şekil 5. İlk → son sürüm kalite değişimi (%).



Şekil 6. Teknik borç ile ilişkili ham metriklerin trendi.

V. TARTIŞMA

Bulgular ilk bakışta çelişkili görünmektedir: TQI yükselirken anlaşılabilirlik ve kapsülleme gibi göstergeler gerilemektedir. Bu çelişki, QMOOD denklemlerinin yapısı incelendiğinde açıklığa kavuşur. Yeniden kullanılabilirlik ve işlevsellik denklemleri büyük ölçüde tasarım boyutu (DSC) ve mesajlaşma (CIS) terimlerinden etkilenir; kod tabanı büyüdükçe bu terimler de büyür ve ilgili nitelikleri yukarı çeker. Dolayısıyla TQI'deki yükseliş, gerçek bir bakım kolaylığı iyileşmesinden çok, sistemin boyutsal büyümesinin bir yan ürünüdür.

Gerçek bakım göstergeleri ise tersi yönde hareket etmektedir. Bağlaşımın artması (+%14,7) ile kohezyonun azalması (-%7,7) bir arada görüldüğünde, bu klasik bir mimari bozulma örüntüsüdür: modüller arası bağımlılık artarken sınıfların tek bir sorumluluğa odaklanması zayıflamaktadır. Kapsüllemenin gerilemesi (-%13,1) iç durumun dışarıya daha fazla açıldığını, anlaşılabilirliğin %30 düşmesi ise sistemin kavranmasının zorlaştığını göstermektedir. Bu örüntü, Lehman'ın artan karmaşıklık ve kalite gerilemesi yasalarıyla [1] ve Parnas'ın yazılım yaşlanması kavramıyla [2] doğrudan örtüşmekte; ortaya çıkan tablo, Cunningham'ın teknik borç eğretilemesi [11] ve bunu izleyen kuramsal çalışmalar [12], [13] ışığında birikmiş bir teknik borç olarak yorumlanabilir.

Ardışık sürüm analizi, 1.20.1 sürümünde TQI'de yerel bir düşüş olduğunu ortaya koymuştur. Bu düşüş, aynı sürümde kapsüllemenin (DAM) en düşük değerine (0,623) inmesi ve

bağlaşımın yükselmesiyle açıklanabilir; sonraki sürümlerde kapsüllemenin kısmen toparlanması ve hiyerarşi sayısının artmasıyla TQI yeniden yükselmiştir. 1.22.1 sürümündeki işlevsellik sıçraması ise hiyerarşi sayısının (NOH) ve tasarım boyutunun artmasıyla ilişkilidir.

VI. LLM DESTEKLİ KALİTE DEĞERLENDİRMESİ

A. Deney Düzenneği ve Kullanılan İstem

Projenin ikinci aşamasında, elde edilen tasarım metriklerinin dört farklı büyük dil modeli — ChatGPT, Gemini, DeepSeek ve Claude Sonnet 4.6 — tarafından nasıl yorumlandığı incelenmiştir. Üç modele de birebir aynı istem (prompt) ve aynı girdi, yani sürümlere göre on bir tasarım özelliğini içeren ham metrik tablosu (Tablo I) verilmiştir. Karşılaştırmanın yanlılığını önlemek amacıyla modellere, bu çalışmada hesaplanan QMOOD kalite skorları verilmemiş; yalnızca ham metrikler sunularak modellerin bağımsız bir değerlendirme üretmesi istenmiştir.

Kullanılan istem üç yinelemeyle son hâline getirilmiştir. İlk sürümde yalnızca “bu metrikleri değerlendir” denmiş; çıktılar yüzeysel kalmış ve modeller bazı kısaltmaları (örneğin yüksek DCC'yi olumlu) yanlış yorumlamıştır. İkinci sürümde metrik tanımları ve “yüksek = iyi/kötü” yönleri eklenerek yorumlar tutarlılaştırılmıştır. Üçüncü ve son sürümde istenen altı analiz başlığı, “yalnızca veriye dayan, uydurma” kısıtı ve yapılandırılmış çıktı biçimi tanımlanarak modeller arası karşılaştırma mümkün kılınmıştır. Modellere verilen son istem aşağıda sunulmuştur:

Kullanılan İstem (dört modele de aynen verilmiştir)

Rol: Nesne yönelimli yazılım tasarımı ve yazılım kalitesi metrikleri konusunda uzman bir yazılım mimarisin.

Görev: jsoup'un 10 ardışık sürümünden (1.13.1 → 1.22.1) statik analizle çıkarılmış nesne yönelimli tasarım metriklerini yorumlayarak tasarım kalitesinin sürümler boyunca nasıl evrildiğini analiz et. Yalnızca verilen sayısal verilere dayan; veride olmayan bilgileri uydurma. Emin olmadığın çıkarımları açıkça belirt.

Metrik tanımları: DSC, NOH, ANA, DAM (yüksek = iyi), DCC (yüksek = kötü), CAM (yüksek = iyi), MOA, MFA, NOP, CIS, NOM (yüksek = daha karmaşık). [Bölüm III'teki tanımların aynı verilmektedir.]

Veri: On bir tasarım özelliğinin sürümlere göre ham değerlerini içeren tablo eklenmiştir (bkz. Tablo I).

İstenen analizler: Her biri ayrı başlık altında ve veriden sayısal örneklerle: (1) Genel kalite değerlendirmesi; (2) Bakım yapılabilirlik analizi (DCC, CAM, NOM, DAM); (3) Teknik borç tahmini (hangi sürümlerde, hangi metrikler); (4) Refactoring önerileri; (5) Mimari kalite yorumları (genişletilebilirlik, soyutlama, modülerlik); (6) Altı QMOOD niteliğinin eğilimi (Artıyor / Azalıyor / Durağan) — tablo biçiminde.

Çıktı biçimi: Altı başlık aynı sırayla; altıncı başlık tablo; sonda 3–4 cümlelik genel sonuç paragrafı.

B. Model Çıktılarının Ayrıntılı Değerlendirmesi

B.1 ChatGPT (eksiksiz değerlendirme). On sürümün tamamını ele alan model, sistemin işlevsel olarak büyüdüğünü (DSC 242→290, +%19,8; NOH 2→4) ancak tasarım göstergelerinde kademeli bozulma yaşandığını belirtmiştir: bağlaşım DCC 4,82→5,52 (+%14,7) ve karmaşıklık NOM 6,09→7,24 (+%18,8) yükselirken kohezyon CAM 0,853→0,788 ve kapsülleme DAM 0,752→0,654 gerilemiştir. Model, soyutlamanın korunduğunu (ANA ≈ 2; MFA ≈ 0,24) ve kompozisyon kullanımının arttığını (MOA 0,90→1,10) doğru biçimde olumlu işaretler olarak ayırt etmiştir. Teknik borcun 1.17.1 sonrasında başladığını, en yoğun bozulmanın 1.20.1 sürümünde (DAM 0,623 ile en düşük) görüldüğünü ve sonraki sürümlerde kısmi toparlanmaya rağmen borcun tümüyle temizlenmediğini saptamıştır. Yeniden düzenleme önerileri olarak Bağımlılığın Ters Çevrilmesi, Extract Class/Method ve kapsüllemenin güçlendirilmesini sıralamıştır.

B.2 Gemini (kısmi veriyle değerlendirme). Bu model, kendisine ulaşan dosya kesitinde yalnızca 1.13.1–1.16.1 sürümlerinin okunabildiğini açıkça belirtmiş ve uydurma yapmama kuralına sadık kalarak çıkarımlarını yalnızca bu dört sürümün eğilimleri üzerine kurmuştur. Bu kısıtlı pencerede dahi DCC 4,81→5,05 artışını, CAM 0,85→0,83 düşüşünü, NOM 6,09→6,52 artışını ve DAM'ın hafif gerilemesini tespit ederek sistemi “şişman sınıf” (fat class) örüntüsüne kayan, monolitikleşen bir yapı olarak nitelemiştir. Soyutlama çekirdeğinin olgun (ANA ≈ 2,01; MFA ≈ 0,24 sabit), çokbiçimliliğin (NOP 2,98→3,19) artışıyla Açık/Kapalı İlkesi'ne uygunluğun korunduğunu vurgulamıştır. Belirleyici kısıt, değerlendirmenin on sürümün yalnızca ilk dördünü kapsamasıdır; dolayısıyla bulguları evrimin tamamını temsil etmez.

B.3 DeepSeek (ayrıntılı, ancak bir metrik hatası içeren değerlendirme). Model on sürümün tamamını ele almış; en önemli bozulmayı kapsüllemeye (DAM 0,751→0,653) ve özellikle 1.16.1→1.17.1 geçişindeki keskin kırılmada (0,741→0,697) görerek 1.17.1'i bir tasarım revizyonu/dönüm noktası olarak işaretlemiştir. Bunu bağlaşım (DCC 4,81→5,52) ve karmaşıklık (NOM 6,09→7,23) artışıyla

birlikte teknik borç birikimi olarak yorumlamıştır. Çokbiçimlilik (NOP 2,98→3,45) ve MFA'daki hafif artışı kalıtımın daha etkin kullanımı olarak olumlu değerlendirmiştir. Ancak model, kompozisyon metriği MOA'yı kohezyon CAM değerleriyle karıştırmış; MOA'nın gerçekte arttığı (0,905→1,103) bir dönemde onu “azalıyor” (0,85→0,79) olarak göstermiş ve etkinlik gerekçesini bu hatalı yorum üzerine kurmuştur. Yine de refactoring önerileri (Tell-Don't-Ask ile kapsülleme, SRP ve arayüzlerle bağlaşım azaltma, ortak davranışın üst sınıflara taşınması) tutarlıdır.

B.4 Claude Sonnet 4.6 (QMOOD skorlarını sayısal hesaplayan değerlendirme). Bu model de on sürümün tamamını ele almış ve diğerleriyle aynı yapısal teşhisi koymuştur: en ciddi gerileme kapsüllemeye (DAM –%13,1), sürekli artış bağlaşım (DCC +%14,7) ve karmaşıklıkta (NOM +%18,8), kademeli düşüş kohezyonda (CAM –%7,7); buna karşılık kompozisyon (MOA +%21,9) ve çokbiçimlilik (NOP +%15,6) olumlu. Teknik borcun en yoğun olduğu dönem 1.17.1–1.20.1 olarak belirlemiş ve 1.19.1→1.20.1 geçişini (en yüksek tek adım DCC artışı +0,148 ve en düşük DAM 0,623) en riskli nokta olarak işaretlemiştir; bu, bu çalışmadaki 1.20.1 TQI düşüşüyle birebir örtüşür. Modelin ayırt edici yönü, altı niteliği yalnızca niteliksel olarak değil, QMOOD ağırlıklandırmasıyla sayısal skorlar üretmek (örneğin yeniden kullanılabilirlik 0,664→0,738) değerlendirmesidir; bu nedenle eğilim kararları bu çalışmanın formül tabanlı QMOOD sonuçlarıyla büyük ölçüde örtüşmüştür.

C. QMOOD Sonuçlarıyla Karşılaştırma

Tablo III, altı kalite niteliğinin eğilimi açısından dört modelin değerlendirmelerini bu çalışmada hesaplanan QMOOD sonuçlarıyla karşılaştırmaktadır. İki nitelikte tam uzlaşma görülmektedir: beş kaynak da anlaşılabilirliğin azaldığı ve işlevselliğin arttığı konusunda hemfikir. Buna karşılık yeniden kullanılabilirlik, esneklik ve etkinlik niteliklerinde ayrışma vardır: QMOOD bu nitelikleri artan olarak hesaplarken ChatGPT, Gemini ve DeepSeek bunları azalan ya da durağan değerlendirmiş; QMOOD ağırlıklandırmasını sayısal uygulayan Claude Sonnet 4.6 ise bu çalışmanın sonuçlarıyla örtüşmüştür.

TABLO III Altı QMOOD kalite niteliğinin eğilimi: bu çalışmanın hesaplaması ile üç dil modelinin değerlendirmesinin karşılaştırması

Nitelik	QMOOD (bu çalışma)	ChatGPT	Gemini	DeepSeek	Claude S. 4.6
Yeniden Kullanılabilirlik	Artıyor	Azalıyor	Azalıyor	Azalıyor	Artıyor
Esneklik	Artıyor	Hafif azalıyor	Durağan	Azalıyor	Hafif artıyor
Anlaşılabilirlik	Azalıyor	Azalıyor	Azalıyor	Azalıyor	Azalıyor
İşlevsellik	Artıyor	Artıyor	Artıyor	Artıyor	Artıyor
Genişletilebilirlik	Durağan	Hafif artıyor	Artıyor	Kararsız	Durağan
Etkinlik	Hafif artıyor	Durağan	Azalıyor	Azalıyor	Durağan

D. Eleştirel Değerlendirme

Tablo III'teki ayrışma, modellerin değerlendirme yöntemiyle yakından ilişkilidir. ChatGPT, Gemini ve DeepSeek metrikleri bir uygulayıcı mühendis sezgisile yorumlamış; artan bağlaşım ve azalan kohezyona ağırlık vererek yeniden kullanılabilirlik, esneklik ve etkinliği gerileyen olarak değerlendirmiştir. Buna karşılık Claude Sonnet 4.6, QMOOD ağırlıklandırmasını sayısal olarak uygulayarak bu nitelikleri — bu çalışmanın formül tabanlı hesabı gibi — artan ya da durağan bulmuştur. Bu durum çalışmanın merkezi gözlemini netleştirir: QMOOD'un yeniden kullanılabilirlik ve işlevsellik denklemleri büyük ölçüde tasarım boyutu (DSC) ve mesajlaşma (CIS) terimlerinden etkilendiğinden, kod tabanı büyüdükçe bu nitelikler mekanik olarak yükselir; sezgisel akıl yürüten modeller ise bu boyut etkisini göz ardı edip gerçek bakım maliyetindeki artışı yakalamaktadır. Dolayısıyla iki yaklaşım çelişmez, birbirini tamamlar.

Modellerin güvenilirliği açısından dört gözlem öne çıkar. Birincisi, Gemini girdiyi tam okuyamamış ve değerlendirmesini yalnızca ilk dört sürüm (1.13.1–1.16.1) üzerinden yapmıştır; bu kısıtı açıkça belirtmesi olumlu olmakla birlikte, kısmi veriye dayanan sonuçları on sürümlük evrimi tam temsil etmez. İkincisi, DeepSeek kompozisyon metriğini (MOA) kohezyon (CAM) değerleriyle karıştırmış; MOA'nın aslında arttığı (0,905 → 1,103) bir dönemde onu azalıyor olarak göstermiş ve etkinlik gerekçesini bu hatalı yorum üzerine kurmuştur. Üçüncüsü, ChatGPT on sürümün tamamını ve MOA dâhil tüm metrikleri doğru yorumlayarak tutarlı bir değerlendirme sunmuştur. Dördüncüsü, Claude Sonnet 4.6 metrikleri doğru yorumlamanın yanı sıra QMOOD skorlarını sayısal olarak üretmek en şeffaf ve izlenebilir değerlendirmeyi sağlamıştır. Bu farklılıklar, dil modeli çıktılarının girdi bütünlüğü ve metrik temellendirmesi açısından mutlaka doğrulanması gerektiğini göstermektedir.

VII. GEÇERLİLİK TEHDİTLERİ

Kurma geçerliliği: QMOOD'un dört tasarım özelliği (CAM, MOA, MFA, NOP) CK çıktısında doğrudan bulunmadığından yaklaşık ölçütlerle hesaplanmıştır. Özellikle kompozisyon (MOA) ve kalıtım (MFA) için kullanılan ölçütler kabadır; bu özellikler, alan tipi ve kalıtılan metot bilgisini doğrudan üreten Designite gibi bir araçla ölçülerek doğrulanabilir; teknik borç ölçüm araçlarının karşılaştırmalı bir değerlendirmesi literatürde mevcuttur [17]. İç geçerlilik: Metrikler tek bir araçtan (CK) elde edilmiştir; ikinci bir araçla çapraz doğrulama, ölçüm yanlılığını azaltacaktır. Normalizasyonun baz sürüme göre yapılması, değerlerin mutlak değil bağıl yorumlanmasını gerektirir. Sonuç geçerliliği: LLM değerlendirmeleri tek seferlik üretimlerdir; modellerin olasılıksal doğası nedeniyle çıktılar tekrarlar arasında değişebilir. Dış geçerlilik: Çalışma tek bir sisteme (jsoup) ve dört modele dayandığından bulgular doğrudan genellenemez; farklı sistemler ve modellerle tekrarlanması gerekir.

VIII. SONUÇ VE GELECEK ÇALIŞMALAR

Bu çalışmada jsoup kütüphanesinin on ardışık sürümü QMOOD modeliyle analiz edilmiş ve tasarım kalitesinin evrimi hem niceliksel olarak hem de dört büyük dil modelinin niteliksel değerlendirmeleriyle ortaya konmuştur. Sonuçlar, sistemin boyut, bağlaşım ve karmaşıklık bakımından büyürken kapsülleme, kohezyon ve anlaşılabilirlik bakımından gerilediğini; TQI'deki görünür artışın boyut bağımlı terimlerden kaynaklandığını ve gerçek bakım göstergelerinin bir teknik borç birikimine işaret ettiğini göstermektedir. Sezgisel akıl yürüten modeller yeniden kullanılabilirlik gibi niteliklerde QMOOD'un aksine bozulma öngörerek gerçek bakım maliyetine dikkat çekmiş; QMOOD ağırlıklandırmasını sayısal uygulayan model ise formül tabanlı sonuçları doğrulamıştır.

Gelecek çalışmalar olarak, yaklaşık hesaplanan tasarım özelliklerinin (özellikle MOA ve MFA) Designite gibi araçlarla doğrulanması, LLM değerlendirmelerinin tekrarlanabilirliğini ölçmek için her modelden birden çok çıktı alınması ve analizin farklı alan ve ölçeklerdeki açık kaynak sistemlere genişletilmesi hedeflenmektedir.

KAYNAKLAR

- [1] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proc. IEEE*, vol. 68, no. 9, pp. 1060–1076, Sep. 1980.
- [2] D. L. Parnas, "Software aging," in *Proc. 16th Int. Conf. Softw. Eng. (ICSE)*, 1994, pp. 279–287.
- [3] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object-oriented design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [4] J. Bansiya and C. G. Davis, "A hierarchical model for object-oriented design quality assessment," *IEEE Trans. Softw. Eng.*, vol. 28, no. 1, pp. 4–17, Jan. 2002.
- [5] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [6] W. Li and S. Henry, "Object-oriented metrics that predict maintainability," *J. Syst. Softw.*, vol. 23, no. 2, pp. 111–122, 1993.
- [7] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Trans. Softw. Eng.*, vol. 22, no. 10, pp. 751–761, Oct. 1996.
- [8] M.-H. Tang, M.-H. Kao, and M.-H. Chen, "An empirical study on object-oriented metrics," in *Proc. 6th Int. Softw. Metrics Symp. (METRICS)*, 1999, pp. 242–249.
- [9] R. G. Dromey, "A model for software product quality," *IEEE Trans. Softw. Eng.*, vol. 21, no. 2, pp. 146–162, Feb. 1995.
- [10] M. M. Lehman, J. F. Ramil, P. D. Wernick, D. E. Perry, and W. M. Turski, "Metrics and laws of software evolution—the nineties view," in *Proc. 4th Int. Softw. Metrics Symp. (METRICS)*, 1997, pp. 20–32.
- [11] W. Cunningham, "The WyCash portfolio management system," in *Addendum to the Proc. Object-Oriented Programming Systems, Languages, and Applications (OOPSLA '92)*, 1992, pp. 29–30.
- [12] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Softw.*, vol. 29, no. 6, pp. 18–21, Nov./Dec. 2012.
- [13] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *J. Syst. Softw.*, vol. 101, pp. 193–220, 2015.
- [14] J. Hedley, jsoup: Java HTML Parser. [Çevrimiçi]. Erişim adresi: <https://jsoup.org/>. Erişim tarihi: 9 Haziran 2026.

- [15] M. Aniche, Java Code Metrics Calculator (CK), 2015. [Çevrimiçi]. Erişim adresi: <https://github.com/mauricioaniche/ck/>. Erişim tarihi: 9 Haziran 2026.
- [16] B. Henderson-Sellers, Object-Oriented Metrics: Measures of Complexity. Upper Saddle River, NJ, USA: Prentice Hall, 1996.
- [17] P. Avgeriou et al., “An overview and comparison of technical debt measurement tools,” IEEE Softw., vol. 38, no. 3, pp. 61–71, May/Jun. 2021.